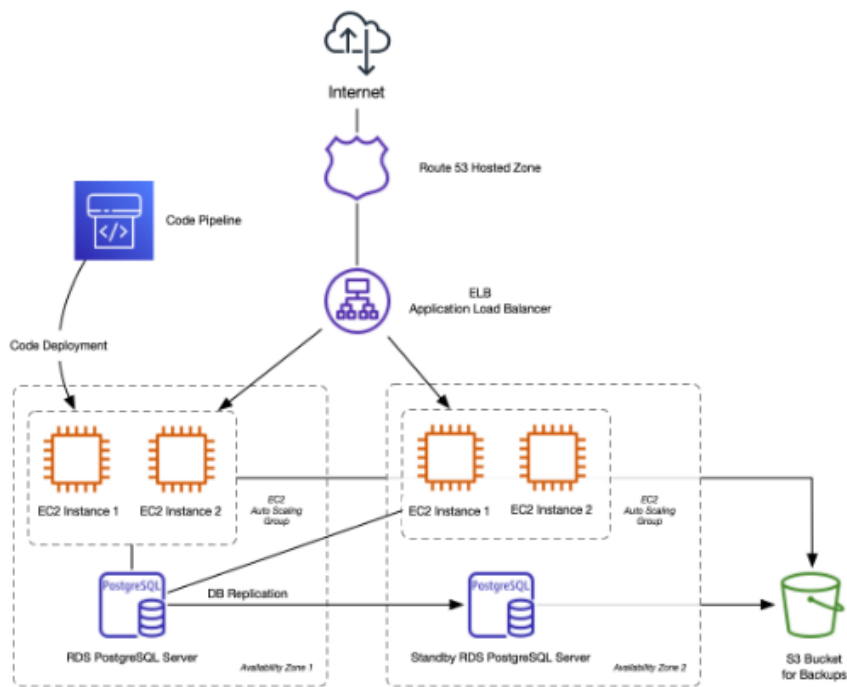


AWS Architecture for Lilly - Explained Simply

Here is the architecture diagram that your team creates for Lilly:



Architecture Diagram

What's the Goal of This Architecture?

The goal is to host a web application (frontend and backend) in a way that is scalable, resilient, and easy to update, especially for a startup.

How Users Access the App:

1. Users connect via the Internet.
2. Route 53 (AWS DNS) handles the domain and directs traffic.

Load Balancing User Requests:

3. Traffic goes to the Elastic Load Balancer (ELB), distributing traffic across servers to avoid overload and downtime.

Running Your Application:

4. The app runs on EC2 instances inside Auto Scaling Groups, across two Availability Zones (for redundancy). AWS scales these up/down as needed.

Code Deployment:

5. CodePipeline automatically deploys new code to the servers whenever updates are made.

Database Layer:

6. RDS PostgreSQL is the main database (in AZ1), with a standby replica (in AZ2) for failover and reliability.

Backup Storage:

7. Amazon S3 stores regular backups of the database.

Component Summary:

- Route 53: DNS, fast routing.
- ELB: Load distribution.
- EC2 + Auto Scaling: Scalable app servers.
- CodePipeline: Automated deployment.
- RDS: Managed PostgreSQL DB.
- RDS Standby: Backup DB.
- S3: Backup storage.

Cost Overview:

- EC2 costs scale with traffic.
- RDS costs vary by size and redundancy.
- S3 charges depend on storage used.
- ELB, Route 53, and CodePipeline have low usage-based costs.

This setup is ideal for startups: it grows with traffic, offers resilience, and minimizes downtime during updates.